

6. Project Testing Phase

Project: PocketSmart AI - Smart Budget & Recommendation Assistant

6.1 Introduction to Project Testing

Purpose of Testing

The Project Testing Phase verifies whether PocketSmart AI works according to its planned functionality. Testing focuses on reliability, accuracy, contextual awareness, budget adherence, platform-related accuracy, and smooth interaction across the Home Interior, Party, and Jewelry planners.

Testing Scope

The documented testing scope covers real-world budget inputs and user preferences across all three planning domains. Testing is intended to check the complete flow from collecting inputs to generating and displaying recommendations.

Testing Objectives

The major objectives are to verify planner functionality, validate user inputs, check Gemini response quality, confirm budget-based recommendations, and ensure that the user interface displays results correctly.

End-to-End Approach

Testing is planned as an end-to-end process. A user provides the required information, the frontend sends the request to the backend, the planner logic prepares the AI request, Gemini generates output, and the result is returned to the appropriate recommendation page.

Expected Testing Result

The testing process should identify functional errors, incomplete outputs, invalid input cases, and situations where recommendations do not sufficiently match the user's requirements.

Page 2 - Home, Party, and Jewelry Testing

Home Interior Testing

Home Planner testing uses different budgets, room types, item quantities, and preferences. Example scenarios include Living Room, Kitchen, and Bedroom requirements with items such as lights, ceiling fans, and dining tables.

Home Recommendation Validation

The generated home recommendations are checked for relevance to the entered requirements and budget. The project documentation states that functionality, style, and price should be considered when generating suggestions.

Party Planner Testing

Party Planner testing uses different guest counts, event types, budgets, and venue details. Scenarios may include birthday, corporate, and wedding planning with food, decoration, entertainment, and accommodation-related needs.

Party Recommendation Validation

Testing checks whether the recommendations reflect the event context and provided budget. Food, venue, and decoration-related results are reviewed as part of the documented real-world testing scope.

Jewelry Planner Testing

Jewelry Planner testing includes budget, occasion, style preference, and optional outfit image inputs. This tests both text-based and image-supported recommendation behavior.

Jewelry Recommendation Validation

The output is checked for alignment with the occasion, style, outfit context, and budget. The image input is considered part of the multimodal testing scenario described in the project.

Page 3 - AI, API, and User Testing

Gemini AI Testing

The project workflow requires sample prompt calls to validate Gemini connectivity. Both text-only and image-plus-text inputs are tested to confirm multimodal compatibility.

Prompt Quality Testing

Prompt structures are evaluated to determine whether the AI understands budget context and planner-specific requirements. When results are weak, the documented workflow calls for prompt optimization.

API Route Testing

The major planner endpoints are tested independently and as part of the complete application flow. Authentication, session, recommendation detail, and history routes are also checked as applicable.

Authentication Testing

Registration, login, logout, token generation, and session handling are tested so that users can access the intended protected functions and personalized features.

Input Validation Testing

Validation testing checks incomplete, invalid, or edge-case inputs before they reach the recommendation logic. The goal is to reduce application failures caused by unsuitable request data.

Fallback Testing

The project workflow includes fallback or default recommendations when the AI returns insufficient results. These fallback behaviors should be tested to make sure the application continues to provide usable output.

Page 4 - UI, Integration, and Optimization Testing

UI Testing

The frontend is tested for correct form behavior, planner navigation, dynamic result display, and card-like recommendation presentation. The interface should remain clear and responsive.

Integration Testing

Integration testing checks communication between the frontend, FastAPI backend, Gemini AI layer, and recommendation services. The complete request-and-response path is verified for each planner.

History and Dashboard Testing

The User Dashboard and Recommendation History functions are checked to ensure that recent recommendations, saved queries, and previous results are displayed as intended.

Cross-Device Usability

The project documentation emphasizes a lightweight, responsive interface. Testing therefore considers whether the main pages and recommendation views remain usable across different screen sizes.

Optimization

Testing results are used to improve prompt structures, validation rules, session handling, and the UI/UX. Optimization is part of the documented final milestone.

Defect Handling

Issues found during testing are grouped by function and corrected before final evaluation. Particular attention is given to budget mismatches, incomplete AI results, invalid inputs, and failed route interactions.

Page 5 - Test Completion and Final Evaluation

Test Completion Criteria

Testing is considered complete when the three planners can process their intended inputs, the AI integration responds correctly, recommendation pages display results, and the major authentication and session functions operate.

Quality Evaluation

The documented evaluation areas include Gemini response quality, budget adherence, platform accuracy, contextual awareness, and consistent performance across the planning categories.

Real-World Scenarios

End-to-end scenarios include home decor setups, party packages involving food, venues, and decoration, and jewelry suggestions based on outfit styles, occasions, and image inputs.

Final Testing Outcome

The final outcome of the testing phase is a validated version of the PocketSmart AI workflow with identified issues addressed through prompt refinement, input validation, and fallback handling.

Conclusion

The Project Testing Phase confirms whether PocketSmart AI performs its intended recommendation and planning functions. By testing every major planner and the supporting application services, the project can move toward final documentation and demonstration with a clearer understanding of its behavior and limitations.